

See to Solve: A Geometry-Aware Vision-Language Agent for Real-World Routing Problem

Wentao Zhang

School of Computer Science and
Engineering, Beihang University
Beijing, China

MOE Engineering Research Center of
Advanced Computer Application
Technology, Beihang University
Beijing, China

Jingyuan Wang

School of Computer Science and
Engineering, Beihang University
Beijing, China

School of Economics and
Management, Beihang University
Beijing, China

Zetong Zhou

School of Computer Science and
Engineering, Beihang University
Beijing, China

MOE Engineering Research Center of
Advanced Computer Application
Technology, Beihang University
Beijing, China

Jiahao Ji*

Meituan Inc.
Beijing, China

MOE Engineering Research Center of
Advanced Computer Application
Technology, Beihang University
Beijing, China

Junjie Wu

School of Economics and
Management, Beihang University
Beijing, China
MIIT Key Laboratory of Data and
Decision Intelligence, Beihang
University
Beijing, China

Abstract

Despite being foundational in logistics, routing problems remain computationally challenging under complex, real-world constraints. Classical heuristics typically lack explicit modeling of data distributions, necessitating expert-driven operator design and extensive manual tuning to ensure algorithmic reliability. Conversely, neural-network-based methods frequently struggle with distribution shifts across diverse urban environments, leading to poor generalization in real-world deployments. To address these dual limitations, we introduce *See to Solve (STS)*, a novel framework that employs a Vision-Language Model (VLM) as an autonomous agent to construct solutions for complex routing problems in an autoregressive manner. At each step, STS integrates multimodal observations to capture geometric structures and constraint-specific signals. To ensure logical grounding and mitigate hallucinations, we enforce an Observation-Thought-Decision reasoning protocol. Our approach further utilizes a two-stage sim-to-real pipeline: supervised fine-tuning on synthetic datasets, followed by GRPO-based reinforcement learning for rapid adaptation to target domains. Evaluation on two real-world routing problems across three cities demonstrates that STS achieves the best overall performance among strong heuristic and neural baselines. Notably, STS exhibits robust cross-city generalization and superior scalability to large-scale instances, bridging the gap between theoretical optimization and real-world deployment. We released our code at <https://github.com/ShotDownDiane/co-agent>.

*Corresponding author: jiahaoji03@gmail.com



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3818894>

CCS Concepts

• Information systems → Spatial-temporal systems.

Keywords

Route planning; Spatial temporal; Large language models

ACM Reference Format:

Wentao Zhang, Jingyuan Wang, Zetong Zhou, Jiahao Ji, and Junjie Wu. 2026. See to Solve: A Geometry-Aware Vision-Language Agent for Real-World Routing Problem. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770855.3818894>

1 Introduction

Routing problems (RPs) are a fundamental class of combinatorial optimization tasks that aim to determine efficient routes for servicing customers over spatial or road-network structures. A solution typically consists of one or multiple tours that start and end at a depot, with the objective of minimizing total travel cost subject to operational constraints. RPs are central to many real-world applications, including logistics distribution [6, 24], on-demand delivery [32, 44], and emergency response [19, 38, 50, 58], where solution quality directly impacts operational cost and service reliability. Due to their combinatorial nature and complex constraint coupling, RPs are NP-hard and remain challenging both in theory and in practice.

The optimization research community has mainly relied on two classes of methods. *Exact methods* formulate RPs as mathematical programs and solve them with general-purpose solvers or exact search procedures [14]. However, exact methods often suffer from rapidly increasing computational cost, and their runtime can become inaccessible as instance size and constraint complexity grow. *Heuristic methods* use hand-crafted rules and iterative improvement

to quickly produce feasible, high-quality solutions, including simulated annealing (SA), ant colony optimization (ACO), etc. However, heuristic methods do not learn reusable decision policies from data. Instead, they construct solutions through search operators and local improvements. In complex scenarios, strong performance depends heavily on expert-crafted operators and careful tuning.

With the rise of neural networks (NNs), routing problems have gained a new family of data-driven solution approaches. By training NNs with supervised learning or reinforcement learning, these methods can learn instance-dependent patterns and decision rules from data, providing an alternative to hand-crafted heuristics and solver-centric pipelines. As a result, NNs are now widely used either as *heuristic components* [45, 55] or as *end-to-end solvers* that directly construct solutions [8, 23, 25]. These neural routing methods have achieved strong performance on standard benchmarks, including the Traveling Salesman Problem [39] and the Capacitated Vehicle Routing Problem [10, 57]. However, this success typically hinges on large-scale in-distribution training data [33, 34]. In practical routing scenarios, such data are typically limited, since obtaining high-quality instances and dependable supervision is expensive. Moreover, standard random instance generation often fails to reflect task-dependent spatial structures and the interdependence between constraints, causing a significant distribution mismatch between synthetic training data and real-world test situations. For example, in the time-dependent traveling salesman problem with time windows (TDTSP-TW), travel costs change over time to reflect urban traffic patterns that fluctuate throughout the day [18, 21, 51]. When combined with time-window constraints, these temporal variations induce complex dependencies in the solution space that simple generators struggle to capture, leading to a substantial simulation-to-reality gap. As a consequence, NNs find it hard to obtain effective supervision or reward signals, which in turn restricts their ability to generalize in real-world settings [20].

We propose to solve real-world routing problems with a vision-language model (VLM) agent. VLMs provide two practical advantages for routing: (i) A unified textual representation provides a flexible interface capable of encoding diverse operational features while preserving complex semantics. By leveraging this expressive medium, the agent can perform high-level reasoning across multiple constraint types within a single prompt; and (ii) Visual observations provide a complementary and intuitive representation of spatial structures compared to flat numerical encodings, explicitly revealing latent geometric patterns that are inherently challenging to extract from raw coordinate data alone. These properties make VLMs a natural backbone for routing under complex constraints and distribution shifts.

Building on this idea, we introduce *See to Solve (STS)*, a training and adaptation framework that turns a general-purpose VLM into a step-by-step routing agent. We formulate routing as an autoregressive decision-making process: at each step, the agent receives a multimodal observation that summarizes the current state and the partial solution, and then selects the next action. To support effective reasoning and reduce hallucinations, we adopt an Observation–Thought–Decision (OTD) format that explicitly grounds each decision in the provided observations. We perform supervised fine-tuning (SFT) in a simulated environment using an OTD-style demonstration dataset to equip the agent with geometric

reasoning and constructive solution-building skills. We then apply reinforcement fine-tuning (RFT) with Group Relative Policy Optimization (GRPO) on target-domain data to align these priors with real-world objectives and distributions, bridging the simulation-to-reality gap. We evaluate STS on two real-world routing tasks across three cities, where it outperforms both classical heuristics and learning-based baselines. Moreover, STS demonstrates strong generalization in both cross-city transfer and large-scale scalability.

Our contributions can be summarized as follows:

- *Domain-specific contributions.* STS reduces the manual engineering effort required by classical heuristics. It replaces complex operator design and parameter tuning with a learned, reusable decision policy. In our real-world benchmarks, STS achieves strong solution quality with runtime comparable to high-budget heuristic methods, while providing interpretable decision traces. To our knowledge, STS is among the first attempts to use a general-purpose VLM as an autoregressive agent for real world routing tasks.
- *Overcoming the challenges of applying neural methods to real-world routing.* We address the simulation-to-reality generalization challenge of neural routing methods by leveraging VLMs as multimodal decision agents. Extensive experiments show that, with only a small amount of target-domain data, STS achieves strong solution quality and generalizes better than prior learning-based baselines in our evaluations. STS further remains robust under both scale increases and cross-city distribution shifts. Finally, its text-based decision outputs provide interpretable explanations, alleviating the black-box nature of conventional neural solvers.

Interdisciplinary Collaboration. This work is a collaboration between Beihang University and Meituan Inc. Domain experts from Meituan provided guidance on problem setting, practical challenges, feature design, and evaluation protocols grounded in real-world scenarios. The Beihang team led the methodology implementation, training pipeline design, and experimental validation and analysis.

2 Related Work

2.1 Classical and Time-Dependent Routing

Routing problems have been extensively studied in operations research, with classical formulations including the traveling salesman problem (TSP) [39] and the vehicle routing problem (VRP) [10]. In practical urban logistics, routing decisions are often further complicated by time-dependent travel costs and service constraints such as time windows [17, 51]. These factors lead to challenging variants such as the time-dependent traveling salesman problem with time windows (TDTSP-TW) and the time-dependent vehicle routing problem with time windows (TDVRP-TW), where edge costs depend on departure time and route feasibility is coupled with temporal constraints [30, 36, 37]. Compared with static benchmarks, these problems exhibit stronger temporal dynamics and tighter constraint coupling, and thus better reflect real-world routing applications [17, 51, 53].

Classical methods for these problems can be broadly divided into exact methods and heuristic or metaheuristic methods. Exact methods, such as branch-and-cut [5], branch-and-bound [1], and constraint programming [41, 48], can provide optimality guarantees, but their computational cost usually grows rapidly with

problem size and constraint complexity. Heuristic and metaheuristic methods, including ant colony optimization (ACO) [11, 40, 42], simulated annealing (SA) [3, 26], and adaptive large neighborhood search (ALNS) [7, 13, 43], are more scalable and can produce high-quality feasible solutions in practice. However, their performance often depends on problem-specific operators, constraint-handling rules, and carefully tuned search budgets. In contrast, our work aims to learn a reusable routing policy that can adapt to real-world urban distributions while preserving interpretable decision traces.

2.2 Learning based Solvers

Learning based solvers provide a data-driven alternative to hand-crafted search rules [25, 28, 29, 49]. Existing methods can be roughly categorized into constructive solvers and improve-based solvers. Constructive solvers sequentially build a solution by selecting the next node or action, which naturally aligns with autoregressive decision making. Pointer networks introduced an early neural architecture for generating permutation-based solutions [49]. The Attention Model (AM) further established a Transformer-style encoder-decoder framework for routing [25], and POMO improved training stability and inference quality by exploiting multiple starting points and problem symmetries [28]. MatNet extends neural routing to asymmetric settings by encoding edge relations in matrix form [29].

Despite their strong performance on standard synthetic benchmarks, neural solvers often rely on abundant in-distribution training instances [25, 28, 29, 49]. Their performance may degrade under real-world distribution shifts, especially when spatial layouts, traffic dynamics, and temporal constraints differ from the synthetic training distribution [20, 52]. Test-time adaptation methods such as Efficient Active Search (EAS) partially address this issue by updating model parameters or search components on target instances [16, 31, 35]. However, these methods still mainly operate on numerical encodings such as coordinates, distance matrices, or edge-weight tensors, and their decision process is usually implicit and difficult to inspect. STS differs from these methods by using multimodal observations to explicitly expose spatial geometry and by producing structured OTD traces for each routing step.

2.3 LLM Agents for Combinatorial Optimization

Recent studies have begun to explore large language models (LLMs) for combinatorial optimization and routing. Existing approaches mainly use LLMs to generate heuristic rules, discover search operators, write solver code, or enhance neural solvers. For example, LEHD-LLM uses LLMs to enhance neural solvers for vehicle routing problems [47], while VRPAgent leverages LLMs to discover heuristic operators for search-based vehicle routing solvers [15]. These methods demonstrate the potential of foundation models for optimization, but they usually treat LLMs as auxiliary components rather than as grounded sequential decision-making agents. Moreover, most of them rely primarily on textual or symbolic inputs and do not explicitly leverage visual spatial observations, which are important for routing problems with geometric structures.

Our work explores a different direction: using a vision-language model (VLM) as an autoregressive routing agent. By combining rendered spatial observations with structured textual constraints, STS allows the model to reason jointly over geometry, travel cost, time

windows, and candidate actions. This design is related to spatial and map representation learning [22, 56], but differs in that STS directly uses multimodal observations for step-by-step routing decisions. We further introduce a program-verified Observation–Thought–Decision protocol and a two-stage SFT–RFT training pipeline, enabling the agent to acquire geometric reasoning skills from simulated tasks and adapt to target-domain urban routing instances.

3 Preliminaries

In this section, we first present the notation and the formal formulation of the routing problem. We then reformulate it as a Markov decision process suitable for VLM agents.

Definition 1 (Customer–Depot Graph). A customer–depot graph G is a complete directed graph defined as $G = (V, E)$, where $V = \{v_0, v_1, \dots, v_n\}$ is the node set and E is the set of edges connecting these nodes. Here, n denotes the number of customers. The customer set is $V_c = V \setminus \{v_0\}$, consisting of n customer nodes, and v_0 denotes the depot.

Definition 2 (Service Routing). A service routing is an ordered sequence of nodes $\tau = (\sigma_0, \sigma_1, \dots, \sigma_n)$ such that $\sigma_0 = v_0$ and $\{\sigma_1, \dots, \sigma_n\}$ is a permutation of V_c . Let c_{ij} denote the travel cost from node v_i to node v_j . The total service cost of τ is

$$\phi(\tau) = \sum_{k=0}^{n-1} c_{\sigma_k \sigma_{k+1}} + c_{\sigma_n \sigma_0}. \quad (1)$$

In realistic urban settings, routing problems are often subject to two common constraints: (i) time-window requirements at locations and (ii) time-dependent travel costs. We briefly describe them below.

Definition 3 (Time-Dependent Traveling Cost). In this setting, the travel cost of traversing a directed edge $(v_i, v_j) \in E$ depends on the departure time from v_i . Specifically, the travel cost is denoted by $c_{ij}(\eta)$, where η is the departure time from v_i . Therefore, the arrival time at v_j is $\eta + c_{ij}(\eta)$.

Definition 4 (Time-Window Constraint). Each node $v_i \in V$ is associated with a time window $[e_i, l_i]$ and a fixed service time s_i , where e_i and l_i denote the earliest and latest allowable service start times, respectively. When traveling to node v_i , the arrival time is determined by the departure time from the previous node and the corresponding travel cost. If the agent arrives before e_i , it must wait until time e_i to start service; if it arrives after l_i , a time-window violation occurs, and a penalty is incurred.

3.1 Real-World Routing Problems

We focus on two complex routing problems as our target tasks: TDTSP-TW and TDVRP-TW. Both tasks capture core decision patterns in real-world on-demand delivery, such as food delivery.

Time-Dependent Traveling Salesman Problem with Time Windows (TDTSP-TW). The TDTSP-TW is defined on a customer–depot graph $G = (V, E)$ with n customers. The objective is to find an optimal service routing τ^* that minimizes a generalized cost comprising travel expenses and time-window penalties. Given the time-dependent nature of the problem, the travel cost for each edge is determined by the departure time. Specifically, the total travel

cost for a routing $\tau = (\sigma_0, \dots, \sigma_n)$ is calculated as:

$$\phi(\tau) = \sum_{k=0}^{n-1} c_{\sigma_k \sigma_{k+1}}(\eta_k) + c_{\sigma_n \sigma_0}(\eta_n), \quad (2)$$

where η_k denotes the departure time from node σ_k . Consequently, the optimal routing τ^* is obtained by solving the following penalty-augmented objective:

$$\tau^* = \arg \min_{\tau} (\phi(\tau) + f(\tau)), \quad (3)$$

where $f(\tau)$ represents the total penalty for time-window violations.

Time-Dependent Vehicle Routing Problem with Time Windows (TDVRP-TW). The TDVRP-TW extends the single-vehicle TDTSP-TW to a multi-vehicle setting. A solution is represented as a set of routes

$$\mathcal{T} = \{\tau^{(1)}, \tau^{(2)}, \dots, \tau^{(M)}\}, \quad (4)$$

where each route $\tau^{(m)} = (\sigma_0^{(m)}, \dots, \sigma_{T_m}^{(m)})$ starts and ends at the depot v_0 . The number of vehicles M is a decision variable (or fixed, depending on the setting), and T_m denotes the length of the m -th route. Valid solutions must ensure that each customer $v_i \in V_c$ is served by exactly one route, satisfying the coverage constraint $\sum_{m=1}^M (T_m - 2) = n$. Consistent with our TDTSP-TW formulation, we treat time windows as soft constraints. With a fixed usage cost $w > 0$ per vehicle, the optimal solution $\mathcal{T}^*$ is obtained by minimizing the total generalized cost:

$$\mathcal{T}^* = \arg \min_{\mathcal{T}} \left(\sum_{m=1}^M (\phi(\tau^{(m)}) + f(\tau^{(m)})) + w \cdot M \right). \quad (5)$$

Here, the objective balances the travel cost ϕ , the time-window penalty f , and the fleet size cost.

3.2 Problem Reformulation

We cast the routing problem as an autoregressive sequential decision-making process. Formally, given an instance defined on graph $G = (V, E)$, the environment state at step t is

$$s_t = (G, \tau_{t-1}), \quad (6)$$

where $\tau_{t-1} = (a_1, \dots, a_{t-1})$ denotes the partially constructed solution. A decision policy π is applied iteratively to extend the partial solution until termination, yielding a complete sequence $\tau = (a_1, a_2, \dots, a_T)$, whose quality is evaluated by the cost $\phi(\tau)$.

In this work, we use a vision-language model (VLM) as the policy model. Rather than feeding the raw state s_t directly into the model, we transform it into a multimodal observation

$$o_t = \text{Transform}(s_t) = \langle I_t, x_t \rangle. \quad (7)$$

Here, I_t denotes a rendered visual input depicting the spatial layout of graph G together with the current partial solution τ_t . x_t provides complementary textual context, including key constraints and a list of admissible candidate actions. We detail the input construction procedure in Section 4.1. Conditioned on the observation o_t , the VLM produces a structured textual output

$$y_t \sim \pi_{\theta}(\cdot | o_t), \quad (8)$$

where π_{θ} denotes the policy parameterized by the VLM. The output follows an *Observation–Thought–Decision* (OTD) format:

$$y_t = \langle \text{OBS}_t, \text{THK}_t, \text{DEC}_t \rangle. \quad (9)$$

Here, OBS_t summarizes the current geometric state and partial solution, THK_t provides grounded reasoning based on the observation and task constraints, and DEC_t specifies the selected action from the candidate set. The decision component DEC_t is then parsed into the executed action a_t , which is applied to the environment to update the state s_{t+1} . Details of this format are provided in Section 4.1.

4 Method

In this section, we present *See to Solve* (STS), a vision-language policy framework for real-world routing problems. As shown in Fig. 1, STS consists of three connected stages: supervised data generation, two-stage training, and inference-time agent loop. The first stage converts expert trajectories into supervised examples that specify how the agent should observe, reason, and act. Using these examples as supervision, the training stage learns a routing policy that makes step-by-step decisions under spatial and temporal constraints. At inference time, the learned policy is rollout autoregressively: each parsed decision updates the environment state, and the process continues until a complete route is constructed.

In the following, we first define the VLM policy interface, including its multimodal inputs and structured OTD outputs. We then describe the data generation process, followed by the two-stage training pipeline and the inference-time rollout procedure.

4.1 VLM Policy Interface

Following the autoregressive formulation in Section 3, STS instantiates the VLM policy interface by converting each routing state into a multimodal prompt and parsing the generated response into an executable action. This interface is used in both supervised data generation and inference-time rollout. It consists of two parts: multimodal observation construction and structured action generation.

Multimodal observation. The input to the VLM is a multimodal prompt $o_t = \langle I_t, x_t \rangle$, where I_t is a rendered visual representation and x_t is a structured textual context. The visual input provides a global view of the spatial layout, while the textual input supplies precise numerical constraints and candidate-level information.

The textual context x_t is organized into three sequential blocks. First, the *task instruction* defines the agent’s role as a constrained routing solver and specifies the required output format. This standardizes the agent’s behavior and requires the response to end with a clear boxed action. Second, the *system status* summarizes the current routing state, including the simulation time η , the agent’s coordinates, the number of unvisited nodes, and task-specific resources such as the remaining vehicle capacity in VRP tasks. We also derive the movement trend from recently visited points to indicate whether the current route is moving toward, away from, or parallel to the depot, which helps the agent identify the current routing stage. Third, the *candidate list* enumerates all feasible actions at the current step. Each candidate node v_j is assigned a unique option identifier, such as “Option A”, and is described with its coordinates, immediate travel cost $c_{ij}(\eta)$, estimated arrival time, time-window slack, traffic cue, and trajectory-alignment information. These features help the model compare candidates under both spatial and temporal constraints. To reduce ambiguity, different context blocks are separated with explicit tags, such as `<Instruction>...</Instruction>`.

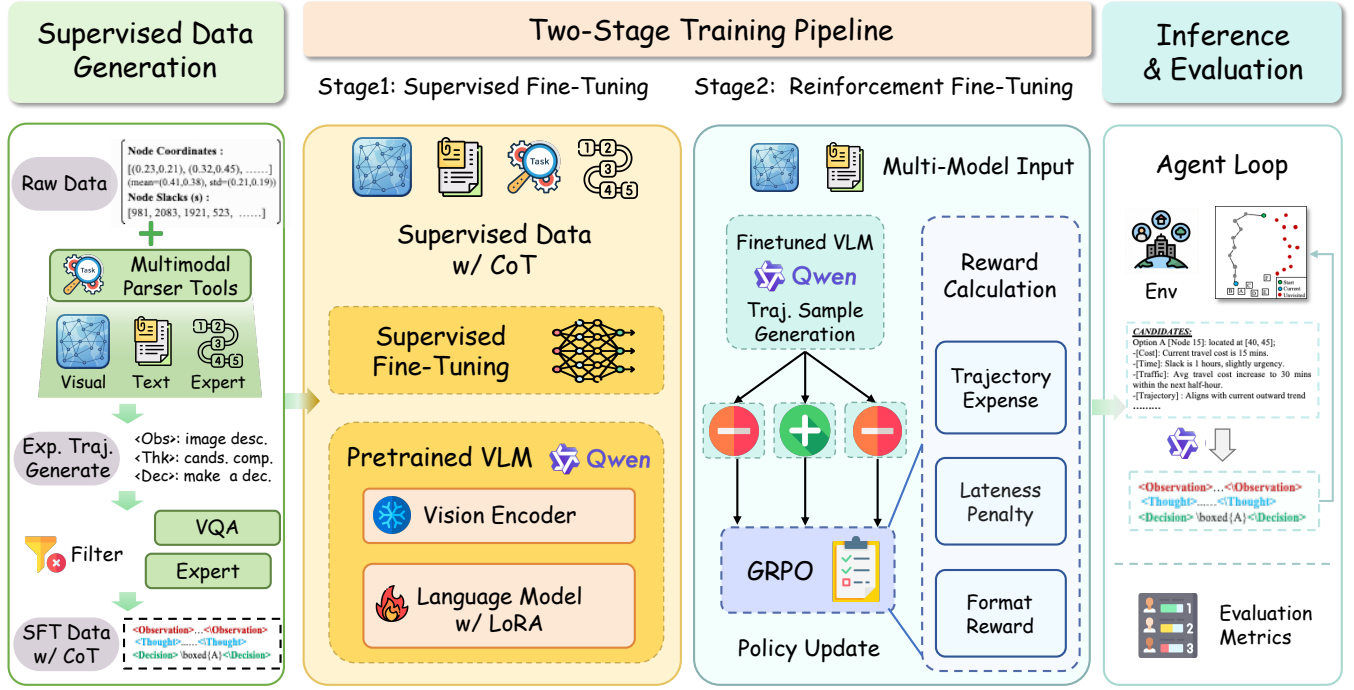


Figure 1: Overview of the STS framework. Left: Program-verified generation of OTD-style supervised data. Center: Two-stage training with SFT and GRPO-based RFT. Right: Inference-time agent loop with multimodal observations and OTD decisions.

The visual input I_t grounds textual reasoning in spatial structure. We render the current routing state as a 224×224 grid image with three layers. The *global geometry* layer plots the depot, current location, and unvisited nodes using the same integer coordinates as those in the textual context, ensuring alignment between visual and textual information. The *trajectory* layer visualizes the decision history τ_{t-1} as a connected path, helping the model perceive the shape and direction of the partial route. The *candidate* layer annotates all feasible candidate nodes with the same option identifiers used in the textual candidate list, enabling the VLM to cross-reference visual patterns with constraints.

Structured OTD response. To make the decision process interpretable and executable, we require the model to generate an Observation–Thought–Decision (OTD) response. The OBS_t field describes visual and geometric cues from the rendered map, such as the spatial distribution of unvisited nodes, the shape of the current trajectory, and the relative positions of candidates. The THK_t field compares feasible candidates by jointly considering geometric patterns and textual constraints, including travel cost, time-window urgency, traffic variation, and trajectory alignment. The DEC_t field gives the final selected action as an option-node pair.

Action parsing. For reliable execution, the decision must be enclosed in a fixed boxed format: $DEC_t = \text{Option } r [\text{Node } j]$, where Option r is an option identifier in the candidate list and v_j is the corresponding candidate node. During rollout, we extract the boxed decision and verify that the option identifier matches the specified node index in the candidate list. If the check succeeds, the corresponding node v_j is executed as the next action; otherwise,

the output is treated as invalid. A detailed and complete prompt example is provided in the Appendix.

4.2 Supervised Data Generation

Given the policy interface defined above, the goal of supervised data generation is to construct training pairs in the same input-output format used by the VLM policy. Specifically, for each decision step, we build a multimodal observation o_t and a supervision label y_t^* . The observation o_t is obtained by applying the interface in Section 4.1 to an expert trajectory state, while the label y_t^* specifies how the agent should observe, reason, and select the expert action. As shown in the left panel of Fig. 1, this process consists of two main steps: expert trajectory generation and program-verified reasoning distillation.

Expert trajectory generation. We first construct expert trajectories on simplified routing tasks, including TSPTW, CVRPTW, and TDTSP-TW/TDVRP-TW, which preserve key spatial and temporal constraints of the target problems. For each instance, a heuristic solver generates a feasible solution recorded as a state-action sequence $\{(s_t, a_t^*)\}_{t=1}^T$, where a_t^* denotes the expert action at step t . Time windows follow the Solomon benchmark protocol, and time-dependent travel times are simulated with a cosine-based speed function to mimic daily traffic fluctuations.

At each step, we retain the expert action a_t^* and construct a feasible candidate set $\mathcal{A}_t$ that includes the expert action and multiple competitive alternatives. The state s_t , together with $\mathcal{A}_t$, is then converted into the multimodal observation o_t using the policy interface described in Section 4.1. This yields intermediate training records of the form $\{(o_t, a_t^*, \mathcal{A}_t)\}_{t=1}^T$.

Table 1: Statistics of the SFT training dataset. We allocate larger sample sizes to fundamental tasks.

Task	Distribution	Node Range (N)	Instances
CVRP-TW	[Uniform, Cluster, Gaussian]	20 – 100	10,000
TSP-TW	[Uniform, Cluster, Gaussian]	20 – 100	10,000
TDTSP-TW	[Uniform, Cluster, Gaussian]	20 – 50	5,000
TDVRP-TW	[Uniform, Cluster, Gaussian]	20 – 50	5,000
Total	Mixed	20-100	30,000

Program-verified reasoning distillation. For each intermediate record, we further construct an OTD-style supervision label y_t^* . Since the final decision is determined by the expert action, the DEC field can be obtained through a deterministic mapping from a_t^* to its corresponding option-node pair in $\mathcal{A}_t$. The main challenge is to generate faithful OBS and THK fields that are consistent with the visual geometry and task constraints. Direct teacher-generated reasoning may contain hallucinations [12, 46], so we use programmatic verification to filter noisy reasoning traces.

We decompose the distillation process into three steps. First, for OBS generation, we define a set of programmable geometric relations covering global distribution patterns, trajectory-aware patterns, and local candidate patterns. Given s_t , a_t^* , and $\mathcal{A}_t$, we check which relations are satisfied or violated, convert these results into contrastive Yes/No questions, and ask the teacher model to answer them before writing OBS _{t} . Only responses whose Yes/No answers are fully consistent with the ground truth are retained.

Second, for THK generation, we prompt the teacher model with the verified OBS _{t} and the textual state summary in o_t . The teacher is instructed to compare the expert action with other candidates from both geometric and constraint-aware perspectives. We generate multiple responses and keep only those that select the expert action a_t^* , using the corresponding reasoning trace as THK _{t} .

Finally, in the refinement step, we provide the teacher model with the verified OBS _{t} , the generated THK _{t} , the current state description, and the expert action a_t^* , and ask it to rewrite them into a clearer and more concise OTD-style trace. The final supervision label is

$$y_t^* = \langle \text{OBS}_t, \text{THK}_t, \text{DEC}_t \rangle, \quad (10)$$

where DEC _{t} is deterministically mapped from a_t^* . The resulting dataset consists of verified training pairs (o_t, y_t^*) , which provide reliable supervision for subsequent supervised fine-tuning.

4.3 Two-Stage Training Pipeline

As shown in the center panel of Fig. 1, to enable effective transfer from geometric reasoning to spatial decision-making, we adopt an imitation-to-reinforcement two-stage training paradigm.

Simulation Dataset. We construct a diverse simulation dataset covering varied problem types and spatial distributions. As detailed in Table 1, the dataset comprises a total of 30,000 instances across four representative routing tasks, ranging from static variants (e.g., CVRP-TW, TSP-TW) to complex time-dependent problems (e.g., TDTSP-TW, TDVRP-TW). To prevent the model from overfitting to specific spatial patterns, we generate node coordinates using a mixture of *Uniform*, *Cluster*, and *Gaussian* distributions, with problem scales

N ranging from 20 to 100. Each instance is paired with an expert-derived solution trajectory, serving as the ground truth for the supervised fine-tuning stage.

Simulation-based Supervised Fine-Tuning (SFT). In the first stage, we perform supervised fine-tuning on the *constructed simulation data* (i.e., the OTD-style CoT corpus). By maximizing the log-likelihood of expert trajectories, the agent learns general geometric perception and step-by-step constructive decision-making via imitation. This stage aims to internalize verified reasoning patterns, providing a stable and generalizable policy initialization before exposing the model to the target environment.

Target-domain Reinforcement Fine-Tuning (RFT). To address the distribution shift, we further optimize the agent on the *target domain instances*. Initialized with the SFT policy, we employ Group Relative Policy Optimization (GRPO) to interact with real-world tasks. As illustrated in the center-right panel of Fig. 1, the reward for each sampled trajectory consists of three components: (1) *Trajectory Expense*: the negative total travel cost of the constructed route, serving as the primary optimization objective; (2) *Lateness Penalty*: for violated trajectories, we define the lateness penalty as $f(\Delta\tau) = \text{clip}(3\Delta\tau, 0.5T, 2T)$, where $\Delta\tau$ is the overtime and T is the total trajectory duration, ensuring that even minor violations incur significant cost while preventing extreme gradient spikes; and (3) *Format Reward*: a binary reward that checks whether the output adheres to the valid OTD structure with a parseable boxed action. The final reward is the sum of these three terms. Unlike standard imitation, this stage allows the agent to explore and directly optimize the target-specific objective without a critic, enabling rapid adaptation to the target data distribution.

4.4 Inference Loop

At inference time, STS directly applies the trained VLM policy to construct routes autoregressively. Starting from the depot, the environment repeatedly builds the feasible candidate set, converts the current state into the multimodal observation defined in Section 4.1, and executes the parsed OTD decision. The rollout terminates when a complete feasible solution is obtained, i.e., all customers are served and the route returns to the depot for TDTSP-TW, or all customers are assigned to vehicle routes for TDVRP-TW. No expert action or teacher-generated reasoning is used during this stage.

5 Experiments

5.1 Experimental Setting

Benchmarks. Following the experimental setting of [54], we conduct experiments on TDTSP-TWD and TDVRP-TW, which span three cities: Berlin, Nairobi, and New York. These cities are selected from Europe, Africa, and America, respectively, to reflect distributional diversity across different continents and urban structures. The instances are grounded in realistic urban routing settings: they are constructed from OpenStreetMap road networks, and time-dependent travel costs are generated from Uber speed data under a FIFO assumption [4]. Customer locations are sampled from real points of interest (POIs), and time windows are generated using a Solomon-style procedure with a mixture of tight and loose settings [9]. Travel times vary by time of day to reflect urban

congestion. For each city, the benchmark includes three large-scale road networks with sizes of 500, 1,000, and 2,000 nodes. From each network, we sample routing instances with $n \in \{20, 50\}$ customers. We ensure that each city contains approximately 500 training instances and 500 test instances; detailed statistics are provided in the Appendix. For all scenarios, we report the objective value (Obj) and the runtime (Time) in seconds.

Baselines and Configurations. We compare STS with baselines from two categories: heuristic solvers and learning-based methods.

For *heuristic solvers*, we include the following baselines: OR-Tools solves routing problems using constraint programming combined with local search and large neighborhood search (LNS). GRASP constructs solutions via greedy randomized insertion with restricted candidate lists. SAH improves GRASP solutions using simulated annealing-based local search; we report both *fast* and *slow* variants with different search budgets. ALNS iteratively destroys and repairs partial solutions using adaptive neighborhood operators, also evaluated under *fast* and *slow* budgets. ACO builds routes by sampling solution components according to pheromone trails and heuristic desirability.

For *learning-based solvers*, we include the following learning-based baselines: AM is an attention-based autoregressive model that constructs routes sequentially by selecting the next node at each step. POMO extends AM with multi-start rollouts to reduce solution variance and improve exploration during inference. TMat is a time-dependent variant of MatNet that encodes routing instances as matrices and is adapted to handle time-dependent travel costs in TDTSP. Following common practice, we further apply Efficient Active Search (EAS) to adapt all neural baselines on the target-domain training set (e.g., AM+EAS). We also evaluate EAS with a randomly initialized policy, denoted as AM*+EAS, to isolate the effect of EAS on target instances. Detailed architectural adaptations and training configurations are provided in the Appendix. Our implementation is based on the rl4co library [2].

For *See to Solve (STS)*, we adopt Qwen3-VL-4B-Instruct as the backbone model and perform SFT using LoRA for parameter-efficient adaptation. LoRA modules are applied to all transformer layers with a rank of 16, scaling factor 32, and dropout 0.05. The maximum sequence length is set to 4096 tokens. SFT is conducted for 2 epochs with a learning rate of $1e-4$. In the subsequent RFT stage, we use a group size of 8 and set the KL coefficient to 0.01 to stabilize policy updates. We provide the full configuration in the Appendix. Moreover, we evaluate STS in two modes: *STS (SFT)*, which is directly evaluated without any target-domain RFT, and *STS (RFT)*, obtained after RFT on the target-domain training set. All experiments are conducted on A800 GPUs with an Intel Xeon(R) Platinum 8470Q CPU. The inference process for STS is served by vLLM [27].

5.2 Overall Performance

Tab. 2 reports the experimental results evaluated on instances of size $n \in \{20, 50\}$. We focus on a data-scarce setting, where learning-based methods are trained using only 500 target-domain instances for each instance size.

(1) STS achieves the best overall performance. Across TDTSP-TW and TDVRP-TW tasks, STS (RFT) achieves the best overall performance and obtains the best or second-best results in most

evaluated settings. The performance gains are stable across instance sizes and cities, indicating that reinforcement fine-tuning enables the model to reliably align its decision policy with target-domain dynamics and optimization objectives. STS (SFT) already exhibits strong performance in most cases, suggesting that the STS framework learns transferable geometric and construction priors from simplified tasks. Overall, these results indicate that the superior performance of STS stems from the combination of transferable knowledge acquired via SFT and target-domain adaptation through RFT, rather than relying solely on training in the target domain.

(2) Limits of heuristics. Heuristic methods are sensitive to search budgets under time-dependent costs and time-window constraints. High-budget variants such as SAH* and ALNS* remain competitive, while their low-budget counterparts degrade noticeably, indicating that sufficient search is needed to handle coupled spatial and temporal constraints. OR-Tools performs well on smaller instances but becomes less competitive on larger ones under fixed time limits. ACO and low-budget GRASP/SAH/ALNS variants are generally weaker, especially as the problem scale increases.

(3) Learning-based baselines show limited transferability. Learning-based methods exhibit large performance gaps across settings, indicating that policies trained mainly on synthetic instances do not fully transfer to target domains with real-world distributions. Although TMat explicitly models time-dependent costs, it performs poorly in our data-scarce setting, suggesting that its inductive bias may not be sufficient when target-domain data are limited. Introducing EAS, such as AM+EAS, improves performance in several cases, showing that test-time adaptation can partially mitigate distribution shift. To further isolate the effect of adaptation, we include AM*+EAS, POMO*+EAS, and TMat*+EAS, where EAS is applied from a randomly initialized policy. In some settings, TMat*+EAS even outperforms TMat+EAS, suggesting that synthetic pretraining may be ineffective or may induce negative transfer under distribution shift and data scarcity.

(4) Efficiency analysis. In terms of runtime, learning-based methods are the fastest due to their low-cost feed-forward inference. STS requires step-by-step VLM decoding and is therefore slower than lightweight neural solvers, but its runtime is comparable to slow-budget heuristic methods. When the problem size increases from $N = 20$ to $N = 50$, the runtime of STS grows moderately and remains within a practical range, indicating its scalability to larger routing instances.

5.3 Generalization

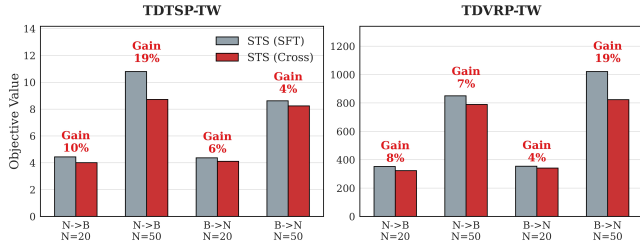
This section evaluates the generalization ability of STS in terms of transferability and scalability.

Cross City. We conduct cross-city generalization experiments to evaluate whether STS can transfer its learned routing policy across different urban environments. This setting is challenging because cities differ in road-network structures, spatial distributions of customer nodes, and time-dependent traffic patterns. Here, *STS (SFT)* denotes the model trained only on simulated data, while *STS (Cross)* denotes a model further adapted via RFT on one source city and directly evaluated on another target city without additional training.

Fig. 2 reports the results across different transfer directions and problem scales. STS (Cross) consistently outperforms STS (SFT),

Table 2: Performance comparison of average on 3-run results. The bold/underlined font means the best/the second-best result.

Method		TDTSP-TW												TDVRP-TW											
		Berlin				Nairobi				New York				Berlin				Nairobi				New York			
		20	50	20	50	20	50	20	50	20	50	20	50	20	50	20	50	20	50	20	50				
		Obj	Time	Obj	Time	Obj	Time	Obj	Time	Obj	Time	Obj	Time	Obj	Time	Obj	Time	Obj	Time	Obj	Time	Obj	Time	Obj	Time
Heuristic	OR-Tools	4.23	60	7.17	120	4.11	60	7.03	120	4.86	60	7.98	120	346	60	708	120	349	60	980	120	598	60	963	120
	GRASP	4.25	5	16.3	14	4.30	6	15.5	15	4.38	5	15.5	13	415	8	1201	12	598	7	1509	12	1020	10	1761	17
	SAH	4.19	13	6.93	35	4.25	15	10.4	35	4.32	11	14.1	32	379	15	1110	37	578	14	1458	36	934	16	1522	28
	SAH*	4.18	23	6.76	108	4.07	33	6.33	94	4.21	21	6.58	126	333	58	804	136	411	33	949	138	633	22	953	125
	ALNS	4.18	14	8.06	78	4.08	12	7.41	79	4.22	16	13.5	68	314	27	938	86	387	27	1067	79	601	24	970	73
	ALNS*	4.15	54	6.44	645	4.01	45	6.20	604	4.09	58	6.11	589	296	64	617	464	338	52	912	503	573	46	876	497
	ACO	5.05	78	11.8	212	4.09	62	11.5	263	4.26	89	12.3	290	314	107	1195	280	365	92	1413	221	662	76	1631	281
Learning	AM	7.47	0.1	13.8	0.2	7.73	0.1	13.9	0.2	7.35	0.1	23.0	0.2	443	0.2	943	0.2	625	0.2	1499	0.2	937	0.2	1425	0.4
	AM* + EAS	6.15	0.1	15.6	0.2	4.76	0.1	15.4	0.3	5.16	0.1	22.2	0.2	569	0.2	1082	0.2	577	0.2	1194	0.2	752	0.2	1315	0.4
	AM+EAS	4.61	0.1	9.62	0.2	5.32	0.1	9.20	0.3	5.51	0.1	21.0	0.2	328	0.2	862	0.2	558	0.2	993	0.2	619	0.2	1244	0.4
	POMO	5.16	0.2	15.0	0.4	5.15	0.3	13.9	0.5	6.95	0.2	17.95	0.3	517	0.3	906	0.4	573	0.3	1150	0.5	625	0.3	1271	0.5
	POMO*+EAS	4.92	0.2	17.3	0.4	5.08	0.3	15.1	0.5	6.21	0.2	15.41	0.4	605	0.3	1219	0.4	565	0.3	952	0.5	829	0.3	1491	0.5
	POMO+EAS	4.56	0.2	12.2	0.3	4.77	0.3	12.3	0.5	5.51	0.2	14.97	0.4	419	0.3	897	0.4	556	0.3	909	0.5	626	0.3	1074	0.5
	TMat	8.43	0.1	14.4	0.2	7.94	0.1	15.6	0.2	7.12	0.1	17.98	0.2	704	0.2	986	0.3	614	0.3	1107	0.3	949	0.3	1577	0.4
	TMat*+EAS	6.55	0.1	12.8	0.3	7.01	0.1	14.9	0.2	6.05	0.1	12.18	0.2	700	0.2	1255	0.3	710	0.3	1530	0.3	894	0.3	1489	0.4
TMat+EAS	6.92	0.1	9.73	0.2	7.56	0.1	10.1	0.2	6.99	0.1	11.92	0.2	618	0.2	934	0.3	601	0.3	1012	0.3	871	0.3	1404	0.4	
STS (SFT)		4.44	67	10.8	148	4.37	52	8.62	173	4.85	55	8.42	161	352	58	850	140	354	83	1021	186	610	57	954	166
STS (RFT)		3.97	55	6.70	142	3.87	44	5.63	158	4.03	54	5.42	153	291	52	624	134	317	71	824	175	542	55	787	158

**Figure 2: Cross-city generalization. STS (SFT) is trained with SFT on simulated data only, STS (Cross) further applies cross-city RFT, and Gain denotes the improvement.**

showing that RFT on one city helps the agent learn routing strategies that transfer beyond the source environment. This suggests that STS captures general decision rules, such as balancing travel cost, time-window urgency, and route geometry, rather than memorizing city-specific patterns. The gains are larger on larger instances, where broader decision spaces make local greedy errors more likely to accumulate. Overall, cross-city RFT improves solution quality and robustness under urban distribution shifts.

Scale. From the main results shown in Tab. 2, we observe that the relative performance between STS and strong heuristic solvers is not monotonic at different scales: STS performs better than ALNS* at $N = 20$, but falls slightly behind it at $N = 50$. To further examine this transition, we perform a fine-grained analysis on Berlin TDVRP-TW across $N \in \{20, 30, 50, 70, 100\}$ and report paired t -test results in Table 3. As shown in Table 3, STS is not uniformly better than ALNS* at small scales. It slightly improves over ALNS* at $N = 20$, performs worse at $N = 30$, and shows a small, statistically insignificant gap at $N = 50$. However, when the scale increases to $N = 70$ and $N = 100$,

Table 3: Comparison between ALNS* and STS on Berlin TDVRP-TW across scales. Lower values are better.

N	ALNS*	STS	Improv.	p -value
20	296	291	+1.69%	0.271
30	483	507	-4.97%	0.032*
50	617	624	-1.13%	0.189
70	916	867	+5.35%	0.014*
100	1802	1252	+30.52%	<0.01**

STS achieves statistically significant improvements. These results suggest that the advantage of STS is less stable on small instances, but becomes clearer as the horizon and decision space grow.

We then extend the evaluation to larger instances with $N \in \{100, 200, 500, 1000, 2000\}$ and compare STS with two strong heuristic baselines in terms of both objective value and runtime. As shown in Fig. 3, STS achieves the best or tied-best solution quality on these large-scale instances. Moreover, its advantage over strong heuristic baselines, especially ALNS*, becomes more pronounced as N increases. In terms of efficiency, the runtime of STS increases with problem size but remains consistently lower than ALNS* and SAH*, showing an approximately linear growth trend. Together, the small-to-medium scale analysis and the large-scale results indicate that STS is effective when the routing problem becomes larger and long-horizon decision errors are more likely to accumulate.

5.4 Ablation

We conduct an ablation study on TDTSP-TW using Berlin instances to examine the contribution of each component. We consider two groups of variants: training-stage and inference-input ablations.

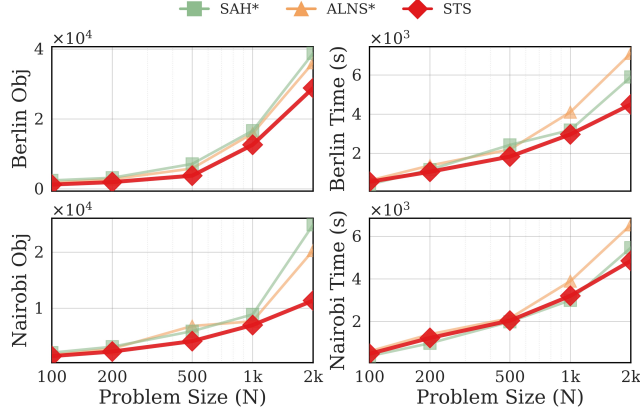


Figure 3: Performance comparison on large-scale instances.

We also provide additional ablations on the rendered image size and inference budget in Appendix.

Ablations on training. (i) *w/o SFT* removes the SFT stage and relies solely on target-domain RFT; (ii) *w/o CoT* trains the model to predict actions only, and uses expert actions as the only supervision signal in the SFT stage; (iii) *w/o Visual* removes the visual input I_t and represents the graph state using coordinate-based text descriptions; and (iv) *naïve agent* is a text-only agent that takes textual inputs and directly outputs actions.

Ablations on inference inputs. We further remove key decision features from the textual dashboard: (v) *w/o Cost* removes candidate travel-cost features; (vi) *w/o Time* removes the current-time context and time-window features; and (vii) *w/o Traffic* removes time-dependent traffic features.

Table 4 shows that removing any component hurts performance, confirming the role of each design choice. The largest drop comes from *w/o SFT*, where the objective increases to 5.25 at $N=20$ and 9.81 at $N=50$. This result shows that simulation-based SFT provides an important initialization for target-domain adaptation, especially when only limited real-world training data are available. Removing reasoning or visual input also leads to clear performance drops. At $N=50$, *w/o CoT* and *w/o Visual* reach 8.23 and 8.02, respectively. This suggests that OTD-style reasoning helps the model compare candidate actions in a more structured way, while visual observations provide useful spatial information beyond coordinate-based text. The *naïve agent* performs worst among the text-only variants, further showing that combining visual perception with structured reasoning is important for effective routing decisions. The inference-input ablations show that the agent also uses the decision features provided in the prompt. Removing cost features causes a relatively smaller drop, while removing time-related information leads to a larger degradation. This is consistent with the nature of TDTSP-TW, where departure time and time-window constraints directly affect the quality of each decision. Removing traffic features also lowers performance, suggesting that dynamic travel-time cues provide useful extra information under time-varying conditions. Overall, these results show that STS benefits from the joint design of simulation-based initialization, visual grounding, structured reasoning, and constraint-aware input features.

Table 4: Ablation study on Berlin. Obj is the objective value (lower is better). $\Delta\%$ is the relative drop compared to STS.

Variants	Berlin ($N=20$)	Berlin ($N=50$)
STS	3.97 (–)	6.70 (–)
<i>Ablations on training</i>		
w/o SFT	5.25 (32%)	9.81 (46%)
w/o CoT	4.38 (10%)	8.23 (23%)
w/o Visual	4.57 (15%)	8.02 (20%)
naïve agent	4.82 (22%)	9.62 (44%)
<i>Ablations on inference inputs</i>		
w/o Cost	4.32 (9%)	7.66 (14%)
w/o Time	4.72 (19%)	8.67 (29%)
w/o Traffic	4.54 (14%)	8.12 (21%)

6 Limitations and Ethical Considerations

Data privacy. Our experiments use only publicly available datasets and therefore do not introduce additional privacy risks beyond those associated with the underlying data.

Limitations. Despite the strong effectiveness of STS, we note several limitations: (1) *Runtime*: our environment must render multi-modal observations and use a VLM as a step-by-step agent, which introduces additional overhead and can lead to longer runtime than lightweight learning-based methods, although we expect this gap to narrow with advances in inference systems and model compression; (2) *Theoretical guarantees*: like most data-driven neural solvers, STS does not provide strict optimality guarantees, although our experiments show stable and quality solutions across multiple runs.

7 Conclusion

In this paper, we study real-world routing problems with complex constraints and limited labeled training data, where existing heuristic and learning-based methods often face challenges in reliable deployment. We propose *See to Solve* (STS), a VLM-based routing agent that constructs solutions through visual and autoregressive decision making. STS learns geometry-aware decision patterns from OTD-style chain-of-thought supervision distilled from simplified tasks through a program-verified reasoning pipeline. We further apply reinforcement fine-tuning on a small target-domain dataset to adapt the agent to real-world objectives and distributions. Experiments on time-dependent routing tasks across multiple cities show that STS achieves strong performance against heuristic and learning-based baselines, while also demonstrating favorable generalization across problem scales and urban environments.

Acknowledgments

Prof. Wang’s work is supported by the National Natural Science Foundation of China (No. 72242101, 72220222, 72171013), Science and Technology Development Fund Macau SAR (0052/2023/RIA1), and the Special Fund for Health Development Research of Beijing (2024-2G-30121). Dr. Junjie Wu’s work was partially supported by the National Natural Science Foundation of China (72242101, 72595861), and the Outstanding Young Scientist Program of Beijing Universities (JWZQ20240201002).

References

- [1] Anna Arigliano, Tobia Calogiuri, Gianpaolo Ghiani, and Emanuela Guerriero. 2018. A branch-and-bound algorithm for the time-dependent travelling salesman problem. *Networks* 72, 3 (Oct 2018), 382–392. doi:10.1002/net.21830
- [2] Federico Berto, Chuanbo Hua, Junyoung Park, Laurin Luttmann, Yining Ma, Fanchen Bu, Jiarui Wang, Haoran Ye, Minsu Kim, Sanghyeok Choi, Nayeli Gast Zepeda, André Hottung, Jianan Zhou, Jieyi Bi, Yu Hu, Fei Liu, Hyeonah Kim, Jiwoo Son, Haeyeon Kim, Davide Angioni, Wouter Kool, Zhiguang Cao, Jie Zhang, Kijung Shin, Cathy Wu, Sungsoo Ahn, Guojie Song, Changhyun Kwon, Lin Xie, and Jinkyoo Park. 2025. RL4CO: an Extensive Reinforcement Learning for Combinatorial Optimization Benchmark. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.
- [3] Dimitris Bertsimas and John Tsitsiklis. 1993. Simulated annealing. *Statistical science* 8, 1 (1993), 10–15.
- [4] Jannis Blauth, Stephan Held, Dirk Müller, Niklas Schlöberg, Vera Traub, Thorben Tröbst, and Jens Vygen. 2024. Vehicle routing with time-dependent travel times: Theory, practice, and benchmarks. *Discrete Optimization* 53 (2024), 100848.
- [5] Jean-François Cordeau, Gianpaolo Ghiani, and Emanuela Guerriero. 2014. Analysis and branch-and-cut algorithm for the time-dependent travelling salesman problem. *Transportation science* 48, 1 (2014), 46–58.
- [6] Patricia J Daugherty, Theodore P Stank, and Alexander E Ellinger. 1998. Leveraging logistics/distribution capabilities: the effect of logistics service on market share. *Journal of Business logistics* 19, 2 (1998), 35.
- [7] Emrah Demir, Tolga Bektaş, and Gilbert Laporte. 2012. An adaptive large neighborhood search heuristic for the pollution-routing problem. *European journal of operational research* 223, 2 (2012), 346–359.
- [8] Darko Drakulic, Sofia Michel, Florian Mai, Arnaud Sors, and Jean-Marc Andreoli. 2023. Bq-nc: Bisimulation quotienting for generalizable neural combinatorial optimization. *arXiv preprint arXiv:2301.03313* (2023).
- [9] Yvan Dumas, Jacques Desrosiers, Eric Gelinas, and Marius M Solomon. 1995. An optimal algorithm for the traveling salesman problem with time windows. *Operations research* 43, 2 (1995), 367–371.
- [10] Burak Eksioğlu, Arif Volkan Vural, and Arnold Reisman. 2009. The vehicle routing problem: A taxonomic review. *Computers & Industrial Engineering* 57, 4 (2009), 1472–1483.
- [11] Casper Joost Eyckelhof and Marko Snoek. 2002. Ant systems for a dynamic TSP: Ants caught in a traffic jam. In *International workshop on ant algorithms*. Springer, 88–99.
- [12] Lizhou Fan, Wenye Hua, Lingyao Li, Haoyang Ling, and Yongfeng Zhang. 2024. Nphardeval: Dynamic benchmark on reasoning ability of large language models via complexity classes. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4092–4114.
- [13] Veaceslav Ghilas, Emrah Demir, and Tom Van Woensel. 2016. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows and scheduled lines. *Computers & Operations Research* 72 (2016), 12–30.
- [14] LLC Gurobi Optimization. 2019. Gurobi Optimization, LLC, Gurobi optimizer reference manual. URL <http://www.gurobi.com> (2019).
- [15] André Hottung, Federico Berto, Chuanbo Hua, Nayeli Gast Zepeda, Daniel Wetzel, Michael Römer, Haoran Ye, Davide Zago, Michael Poli, Stefano Massaroli, et al. 2025. VRPAgent: LLM-Driven Discovery of Heuristic Operators for Vehicle Routing Problems. *arXiv preprint arXiv:2510.07073* (2025).
- [16] André Hottung, Yeong-Dae Kwon, and Kevin Tierney. 2022. Efficient Active Search for Combinatorial Optimization Problems. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=nO5caZwFwYu>
- [17] Jiahao Ji, Jingyuan Wang, Chao Huang, Junjie Wu, Boren Xu, Zhenhe Wu, Junbo Zhang, and Yu Zheng. 2023. Spatio-Temporal Self-Supervised Learning for Traffic Flow Prediction. In *Proc. of AAAI*. 4356–4364.
- [18] Jiahao Ji, Jingyuan Wang, Zhe Jiang, Jiawei Jiang, and Hu Zhang. 2022. STDEN: Towards Physics-Guided Neural Networks for Traffic Flow Prediction. In *Proc. of AAAI*. 4048–4056.
- [19] Jiahao Ji, Jingyuan Wang, Junjie Wu, Boyang Han, Junbo Zhang, and Yu Zheng. 2022. Precision CityShield against Hazardous Chemicals Threats via Location Mining and Self-Supervised Learning. In *Proc. of KDD*. 3072–3080.
- [20] Jiahao Ji, Wentao Zhang, Jingyuan Wang, and Chao Huang. 2025. Seeing the Unseen: Learning Basis Confounder Representations for Robust Traffic Prediction (KDD '25). 577–588.
- [21] Jiawei Jiang, Chengkai Han, Wayne Xin Zhao, and Jingyuan Wang. 2023. PDFormer: Propagation Delay-Aware Dynamic Long-Range Transformer for Traffic Flow Prediction. In *Proc. of AAAI*. 4365–4373.
- [22] Jiawei Jiang, Dayan Pan, Houxing Ren, Xiaohan Jiang, Chao Li, and Jingyuan Wang. 2023. Self-Supervised Trajectory Representation Learning with Temporal Regularities and Travel Semantics. In *ICDE*. 843–855.
- [23] Minsu Kim, Junyoung Park, and Jinkyoo Park. 2022. Sym-nc: Leveraging symmetry for neural combinatorial optimization. *Advances in Neural Information Processing Systems* 35 (2022), 1936–1949.
- [24] Grigoris D Konstantakopoulos, Sotiris P Gayialis, and Evripidis P Kechagias. 2022. Vehicle routing problem and related algorithms for logistics distribution: A literature review and classification. *Operational research* 22, 3 (2022), 2033–2062.
- [25] Wouter Kool, Herke van Hoof, and Max Welling. [n. d.]. Attention, Learn to Solve Routing Problems!. In *International Conference on Learning Representations*.
- [26] Odysseas T Kosmas and Dimitrios S Vlachos. 2012. Simulated annealing for optimal ship routing. *Computers & Operations Research* 39, 3 (2012), 576–581.
- [27] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [28] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai Min. 2020. Pomo: Policy optimization with multiple optima for reinforcement learning. *Advances in Neural Information Processing Systems* 33 (2020), 21188–21198.
- [29] Yeong-Dae Kwon, Jinho Choo, Iljoo Yoon, Minah Park, Duwon Park, and Youngjune Gwon. 2021. Matrix encoding networks for neural combinatorial optimization. *Advances in Neural Information Processing Systems* 34 (2021), 5138–5149.
- [30] Feiyue Li, Bruce Golden, and Edward Wasil. 2005. Solving the time dependent traveling salesman problem. In *The Next Wave in Computing, Optimization, and Decision Technologies*. Springer, 163–182.
- [31] Yang Li, Jinpei Guo, Runzhong Wang, and Junchi Yan. 2023. T2t: From distribution learning in training to gradient search in testing for combinatorial optimization. *Advances in Neural Information Processing Systems* 36 (2023), 50020–50040.
- [32] Yile Liang, Haocheng Luo, Haining Duan, Donghui Li, Hongsen Liao, Jie Feng, Jiuxia Zhao, Hao Ren, Xuetao Ding, Ying Cha, et al. 2024. Meituan's real-time intelligent dispatching algorithms build the world's largest minute-level delivery network. *INFORMS Journal on Applied Analytics* 54, 1 (2024), 84–101.
- [33] Zehua Liu, Zimeng Li, Jingyuan Wang, and Yue He. 2024. Full Bayesian Significance Testing for Neural Networks. In *Proc. of AAAI*.
- [34] Zehua Liu, Zimen Li, Jingyuan Wang, and Yue He. 2024. Full Bayesian Significance Testing for Neural Networks in Traffic Forecasting. In *Proc. of AAAI*.
- [35] Fu Luo, Xi Lin, Fei Liu, Qingfu Zhang, and Zhenkun Wang. 2023. Neural combinatorial optimization with heavy decoder: Toward large scale generalization. *Advances in Neural Information Processing Systems* 36 (2023), 8845–8864.
- [36] Chryssi Malandraki and Mark S Daskin. 1992. Time dependent vehicle routing problems: Formulations, properties and heuristic algorithms. *Transportation science* 26, 3 (1992), 185–200.
- [37] Chryssi Malandraki and Robert B Dial. 1996. A restricted dynamic programming heuristic algorithm for the time dependent traveling salesman problem. *European Journal of Operational Research* 90, 1 (1996), 45–55.
- [38] Balakrishnan S Manoj and Alexandra Hubenko Baker. 2007. Communication challenges in emergency response. *Commun. ACM* 50, 3 (2007), 51–53.
- [39] Rajesh Matai, Surya Prakash Singh, and Murari Lal Mittal. 2010. Traveling salesman problem: an overview of applications, formulations, and solution approaches. *Traveling salesman problem, theory and applications* 1, 1 (2010), 1–25.
- [40] Michalis Mavrouniotis and Shengxiang Yang. 2013. Ant colony optimization with immigrants schemes for the dynamic travelling salesman problem with traffic factors. *Applied Soft Computing* 13, 10 (2013), 4023–4037.
- [41] Penélope Aguiar Melgarejo, Philippe Laborie, and Christine Solnon. 2015. A time-dependent no-overlap constraint: Application to urban delivery problems. In *International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research*. Springer, 1–17.
- [42] Leonor Melo, Francisco Pereira, and Ernesto Costa. 2013. Multi-caste ant colony algorithm for the dynamic traveling salesperson problem. In *International Conference on Adaptive and Natural Computing Algorithms*. Springer, 179–188.
- [43] Stefan Ropke and David Pisinger. 2006. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation science* 40, 4 (2006), 455–472.
- [44] Arianna Seghezzi, Matthias Winkenbach, and Riccardo Mangiaracina. 2021. On-demand food delivery: a systematic literature review. *The International Journal of Logistics Management* 32, 4 (2021), 1334–1355.
- [45] Zhiqing Sun and Yiming Yang. 2023. Difusco: Graph-based diffusion solvers for combinatorial optimization. *Advances in neural information processing systems* 36 (2023), 3706–3731.
- [46] Jianheng Tang, Qifan Zhang, Yuhan Li, Nuo Chen, and Jia Li. 2025. GraphArena: Evaluating and Exploring Large Language Models on Graph Computation. 2025 (2025), 48118–48145.
- [47] Cong Dao Tran, Quan Nguyen-Tri, Huynh Thi Thanh Binh, and Hoang Thanh-Tung. 2025. Large language models powered neural solvers for generalized vehicle routing problems. In *Towards Agentic AI for Science: Hypothesis Generation, Comprehension, Quantification, and Validation*.
- [48] Pascal Van Hentenryck and Vijay Saraswat. 1996. Strategic directions in constraint programming. *ACM Computing Surveys (CSUR)* 28, 4 (1996), 701–726.
- [49] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. 2015. Pointer networks. *Advances in neural information processing systems* 28 (2015).
- [50] Jingyuan Wang, Chao Chen, Junjie Wu, and Zhang Xiong. 2017. No Longer Sleeping with a Bomb: A Duet System for Protecting Urban Safety from Dangerous Goods. In *Proc. of KDD*.

- [51] Jingyuan Wang, Qian Gu, Junjie Wu, Guannan Liu, and Zhang Xiong. 2016. Traffic Speed Prediction and Congestion Source Exploration: A Deep Learning Method. In *Proc. of ICDM*. 499–508.
- [52] Jingyuan Wang, Xu He, Ze Wang, Junjie Wu, Nicholas Jing Yuan, Xing Xie, and Zhang Xiong. 2018. CD-CNN: A Partially Supervised Cross-Domain Deep Learning Model for Urban Resident Recognition. In *Proc. of AAAI*.
- [53] Jingyuan Wang, Ke Tang, Kai Feng, Xin Lin, Weifeng Lv, Kun Chen, and Fei Wang. 2021. Impact of temperature and relative humidity on the transmission of COVID-19: a modelling study in China and the United States. *BMJ open* 11, 2 (2021), e043863.
- [54] Ruixiao Yang and Chuchu Fan. 2025. Neural Combinatorial Optimization for Time-Dependent Traveling Salesman Problem. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [55] Haoran Ye, Jiarui Wang, Zhiguang Cao, Helan Liang, and Yong Li. 2023. DeepACO: Neural-enhanced ant systems for combinatorial optimization. *Advances in neural information processing systems* 36 (2023), 43706–43728.
- [56] Wentao Zhang, Jingyuan Wang, Yifan Yang, and Leong Hou U. 2025. VecCity: A Taxonomy-Guided Library for Map Entity Representation Learning. *Proceedings of the VLDB Endowment* 18, 8 (2025), 2575–2588.
- [57] Jianan Zhou, Zhiguang Cao, Yaoxin Wu, Wen Song, Yining Ma, Jie Zhang, and Chi Xu. 2024. MVMoE: Multi-Task Vehicle Routing Solver with Mixture-of-Experts. *Proceedings of Machine Learning Research* 235 (2024), 61804–61824.
- [58] Konstantinos G Zografos and Konstantinos N Androutsopoulos. 2008. A decision support system for integrated hazardous materials routing and emergency response decisions. *Transportation Research Part C: Emerging Technologies* 16, 6 (2008), 684–703.

A Appendix

A.1 Geometric Pattern

As shown in Tab. 5, we use geometric relations as verifiable primitives for constructing OTD-style Chain-of-Thought (CoT). Instead of relying on free-form descriptions that may be hallucinated, each relation is converted into a verifiable question and checked against programmatically computed results. These relations are organized into three levels: *Global* relations capture instance-level spatial patterns, such as clustered, uniform, or outlier-dominated layouts; *Trajectory* relations describe route geometry, including movement trend, convex-hull status, concavity, nearest-neighbor choices, and isolated pickups; and *Location* relations characterize local candidate context, such as dense zones, spatial holes, boundary nodes, and possible path intersections. These verifiable cues help align STS’s textual reasoning with the underlying spatial topology.

A.2 Prompt Template Example

To make the policy interface more concrete, we provide a representative prompt template used by STS during inference-time rollout. The prompt consists of a visual input I_t and a structured textual context x_t , where the textual context is divided into task instruction, system status, and candidate list. The model is required to return an Observation–Thought–Decision (OTD) response, and the final decision must be written in a boxed format so that it can be directly parsed by the environment. For clarity, Fig. 4 shows a compact single-step example for a TDTSP-TW instance. In implementation, the visual map is provided as an image input together with the textual prompt.

A.3 Details about dataset

We consider three cities (Berlin, Nairobi, and New York) under a congestion-driven, time-dependent setting, and instantiate multiple problem scales with node counts of 500, 1,000, and 2,000. For each city, we simulate travel times from 15:00 to 21:00 with a 10-minute interval, and report the evaluated instance sizes of 20 and 50. To

Template Prompt for STS Agent Rollout	
Panel 1: Task Instruction You are a constrained routing solver. Given the rendered map and the textual state information, select the next node from the feasible candidate list. You should reason about spatial geometry, time-window urgency, traffic variation, and route progress. Your response must follow the Observation–Thought–Decision format and end with a boxed action: Option X [Node Y] .	
Panel 2: System Status Task: TDTSP-TW. Current time: 15:40. Current node: Node 4 at (92, 118). Depot: Node 0 at (112, 104). Partial route: Node 0 → Node 3 → Node 4. Unvisited nodes: {5, 7, 9, 12}. Movement trend: moving northeast and slightly away from the depot.	
Panel 3: Candidate List Option A [Node 5]: coord. (104, 136); travel cost 8.2; arrival time 15:48; slack 34 min; traffic cue: stable; trajectory alignment: aligned. Option B [Node 7]: coord. (130, 142); travel cost 11.5; arrival time 15:52; slack 9 min; traffic cue: increasing; trajectory alignment: aligned. Option C [Node 9]: coord. (70, 95); travel cost 7.8; arrival time 15:48; slack 41 min; traffic cue: stable; trajectory alignment: opposite.	
Panel 4: Visual Input Image: I_t , a 224×224 rendered map showing the depot, current node, unvisited nodes, partial route, and candidate labels A/B/C. The candidate labels in the image are the same as those in the textual candidate list.	
Panel 5: Model Response <Observation> The unvisited nodes are distributed to the northeast and west of the current node. Option B is farther than Option A, but it follows the current movement direction and has a much tighter time window. Option C is closer, but it moves opposite to the current route trend. </Observation> <Thought> Option A has a lower travel cost and enough slack, so it is not urgent. Option B has only 9 minutes of slack and its travel cost may increase soon. Visiting Option B now reduces the risk of a time-window violation while keeping the route direction consistent. </Thought> <Decision> Option B [Node 7] </Decision>	

Figure 4: A single-step prompt example for STS agent rollout.

construct smaller datasets, we select the 200 most congested nodes according to the average edge-weight variance over time, and then sample instances from this subset. Moreover, to ensure the existence of violation-free trajectories in TDTSP-TW, we only sample instances from trajectories that are known to be feasible.

A.4 Baselines Configuration.

We compare STS against two baseline families: heuristic solvers and learning-based methods.

Heuristic solvers. OR-Tools is run with a fixed per-instance time limit. GRASP performs 100 randomized constructions with a restricted candidate list of the top- k insertions ($k=3$), and we keep the best solution as the initializer. SAH (simulated annealing) is initialized by GRASP and evaluated under two budgets: a *fast* setting with 1,000 SA iterations and at most 1,000 local-search calls, and a *slow* setting with 10,000 SA iterations and up to 3,000 local-search calls (denoted as SAH*). ALNS is also initialized by GRASP and run with a maximum of 1,000 iterations (*fast*) or 5,000 iterations (*slow*). ACO uses 30 ants and runs for up to 5,000 iterations.

Learning-based methods. AM (attention-based autoregressive construction) is applied to all settings. MatNet uses matrix encoding to handle asymmetric/time-dependent routing constraints. For both AM and MatNet, we train for 1,000 epochs, with 10,000 synthetic instances per epoch. Unless otherwise stated, the embedding dimension is 256, the number of attention layers is 3, the number of heads is 8, and the feed-forward dimension is 512. For Efficient Active Search (EAS), we adapt each neural baseline on a target-domain

Table 5: Geometric relations used for program-verifiable reasoning distillation.

Category	Verifiable geometric relations
Global	Clustered : whether nodes form dense spatial groups; Uniform : whether nodes are evenly distributed; Outlier : whether isolated nodes are far from the main distribution.
Trajectory	Directional Trend : whether the partial route is moving outward, returning, or parallel to the depot; On Convex Hull : whether a candidate lies on the outer boundary; Concave Region : whether a candidate lies in an indented region; Nearest Neighbor : whether a candidate is closest to the current node; Isolated Pickup : whether an isolated candidate should be visited early to avoid detours.
Location	High-density Zone : whether a candidate lies in a dense remaining-request region; Spatial Hole : whether selecting a candidate would leave a sparse gap; Boundary Node : whether a candidate lies on the periphery of unvisited nodes; Path Intersection : whether selecting a candidate may induce edge crossings.

set of 500 instances. We use a batch size of 50 and run EAS for 20 adaptation steps. The learning rate is 1×10^{-4} .

A.5 Model Comparisons

Table 6 summarizes three additional comparisons on Berlin. First, we evaluate several SOTA VLMs under our multimodal decision framework, where stronger visual reasoning generally leads to better routing decisions. Second, applying the STS pipeline to GLM-4V-9B substantially improves its performance, showing that the framework can transfer across VLM backbones. Third, applying SFT and RFT to AM, POMO, and TMat improves these learning baselines, but none surpasses STS, suggesting that STS benefits not only from the training pipeline but also from multimodal geometry-aware reasoning.

Table 6: Model comparisons in Berlin. Group (1): SOTA VLMs evaluated under our framework. Group (2): backbone generalization. Group (3): SFT/RFT applied to learning baselines.

Method	TDTSP TW-20	TDTSP TW-50	TDVRP TW-20	TDVRP TW-50
<i>(1) SOTA Vision-Language Models</i>				
GLM-4.6V	4.52	8.36	382	803
GPT-5.2	3.99	6.81	298	622
Gemini-3	3.84	6.79	264	609
<i>(2) Backbone Generalization (GLM-4V-9B)</i>				
GLM (base)	10.81	40.42	833	1277
GLM-SFT	4.86	7.02	312	751
GLM-SFT+RFT	4.03	5.93	301	627
<i>(3) Training Pipeline on Learning Baselines</i>				
AM (SFT)	6.69	23.3	430	887
AM (SFT+RFT)	4.28	13.5	330	676
AM (RFT)	7.47	13.8	443	943
POMO (SFT)	5.96	26.2	424	946
POMO (SFT+RFT)	7.07	15.0	371	683
POMO (RFT)	7.07	17.6	517	906
TMat (SFT)	8.61	16.1	433	918
TMat (SFT+RFT)	8.53	13.9	362	679
TMat (RFT)	8.43	14.4	704	986
STS (SFT)	4.44	10.8	352	850
STS	3.97	6.70	291	624

A.6 Additional Ablation Studies

We provide two additional ablation studies to examine the sensitivity of STS to inference budget and visual resolution. The first study

Table 7: Effect of reasoning budget on objective value and runtime on Berlin. Lower objective values are better.

Task	Metric	128	512	1024	4096
TDTSP-TW-20	Obj.	4.35	4.01	3.89	3.97
	Time	18	29	49	55
TDTSP-TW-50	Obj.	7.60	6.82	6.75	6.70
	Time	55	78	143	142
TDVRP-TW-20	Obj.	340	297	289	291
	Time	21	32	47	52
TDVRP-TW-50	Obj.	780	675	634	624
	Time	65	89	122	134

Table 8: Effect of image resolution on Berlin TDVRP-TW. Lower objective values are better.

Setting	112 × 112	224 × 224	448 × 448
$N = 1000$ Obj.	13979	12627	12802
$N = 2000$ Obj.	33108	28870	26001

analyzes the quality–speed trade-off under different reasoning-token budgets, while the second studies how image resolution affects performance in dense large-scale instances.

Reasoning budget. We vary the maximum reasoning budget from 128 to 4,096 tokens and evaluate the objective value and runtime on Berlin. As shown in Tab. 7, a larger budget improves solution quality but also increases inference time. The 1,024-token setting achieves performance close to the default 4,096-token setting, while using 512 tokens reduces runtime by about 41% on average with only a 3.3% increase in objective. This suggests that STS can trade reasoning length for efficiency with limited quality loss.

Image resolution. We further study the impact of visual resolution in dense large-scale settings. Since STS relies on rendered maps to perceive spatial layouts, insufficient resolution may cause nearby nodes to overlap visually. We evaluate three resolutions, 112×112 , 224×224 (default), and 448×448 , on Berlin TDVRP-TW with $N = 1000$ and $N = 2000$. The results are shown in Table 8.

The effect of resolution is moderate at $N = 1000$, where the default 224×224 setting performs slightly better than 448×448 . At $N = 2000$, however, higher resolution becomes more important. The low-resolution 112×112 setting suffers from severe node overlap, with an overlap ratio of 38%, compared with 9% and 2% under 224×224 and 448×448 , respectively. This explains the large performance gap on dense instances and shows that sufficient visual resolution is important for preserving spatial distinguishability.